

Putting Vision to Work

Computer Vision Applications · AI and Machine Learning · Week 2, Day 3

Where we left off

Last class we learned how a machine sees. Three ideas to carry forward:

Image = numbers

A picture is just a grid of pixel values.

Plain net struggles

Too many weights, blind to position, loses layout.

CNN is the tool

Slides small shared filters across the image.

Today: finish transfer learning, look inside a vision CNN, then meet its three big application domains.

Image classification, object detection and semantic segmentation, plus how a CNN is designed for each.



Part 1

Transfer Learning: Borrow a Network That Can Already See

Training a CNN from scratch is expensive

1

Millions of images

needed to learn good filters

2

Many GPUs

heavy computing power

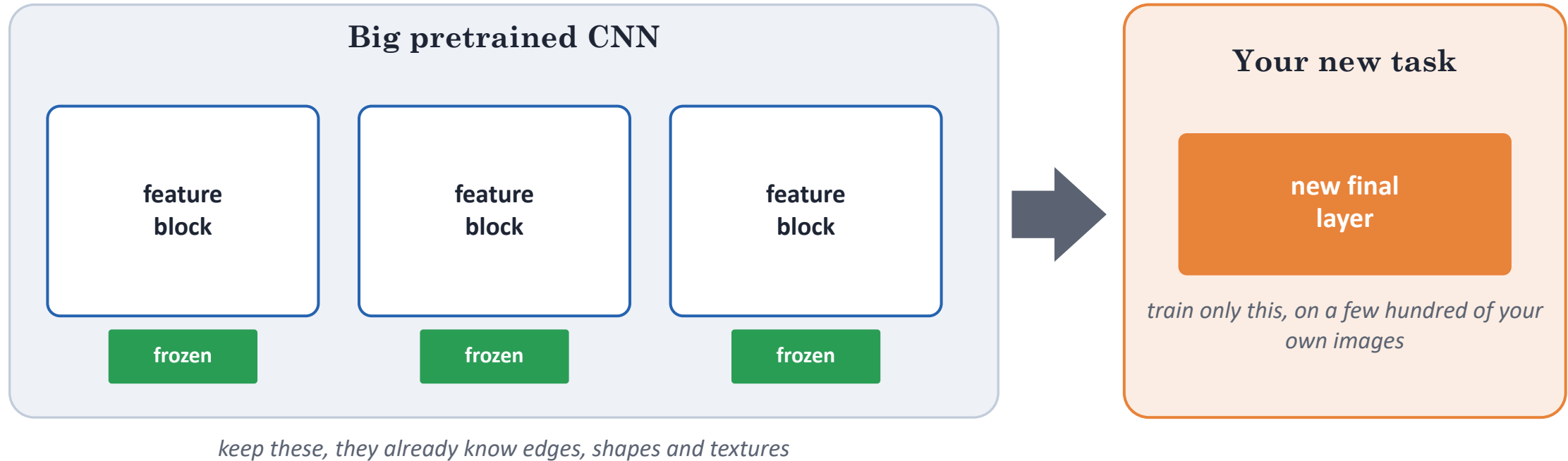
3

Days of training

slow and costly

Your lab (whatever environment you're working in) may not have millions of labelled images, so instead of building from zero, we borrow.

Borrow a network that already learned to see



Reuse the part that already learned to see. Replace only the last layer with your own labels.

Like hiring an expert

The expert already knows how to see

Years of experience reading edges, shapes and textures in any image. You do not pay for that training again.

You only teach your own labels

A quick lesson on your specific problem, a few hundred examples, not years of school.



Think *what is one skill you already have that makes a new, related task much faster to pick up?*

Why transfer learning is perfect for you



Small dataset is enough

A few hundred images can work.



Trains in minutes

Not days, often on a laptop.



Free to start

Many pretrained models are open and free.

An architecture student with 300 facade photos, or a few hundred satellite tiles or microscope images, all of these are possible.

Transfer learning in your field

Environmental Science

A few hundred labelled satellite tiles: forest vs cleared land.

Materials Science

A small set of microscope images tagged sound vs cracked.

Architecture

Around 300 facade photos sorted by building style.

Computer Engineering

A handful of chip or board images per defect type.

Chemical Engineering

A few hundred sample photos labelled by reaction state.



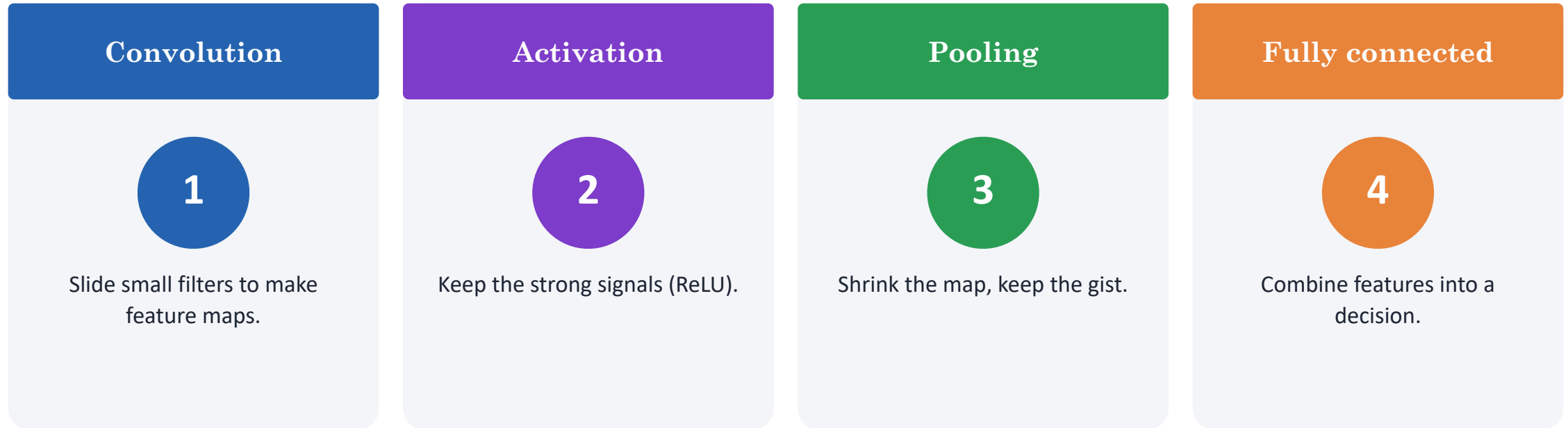
Think *how many labelled images could you realistically collect for a project (maybe your final one) in your own field?*



Part 2

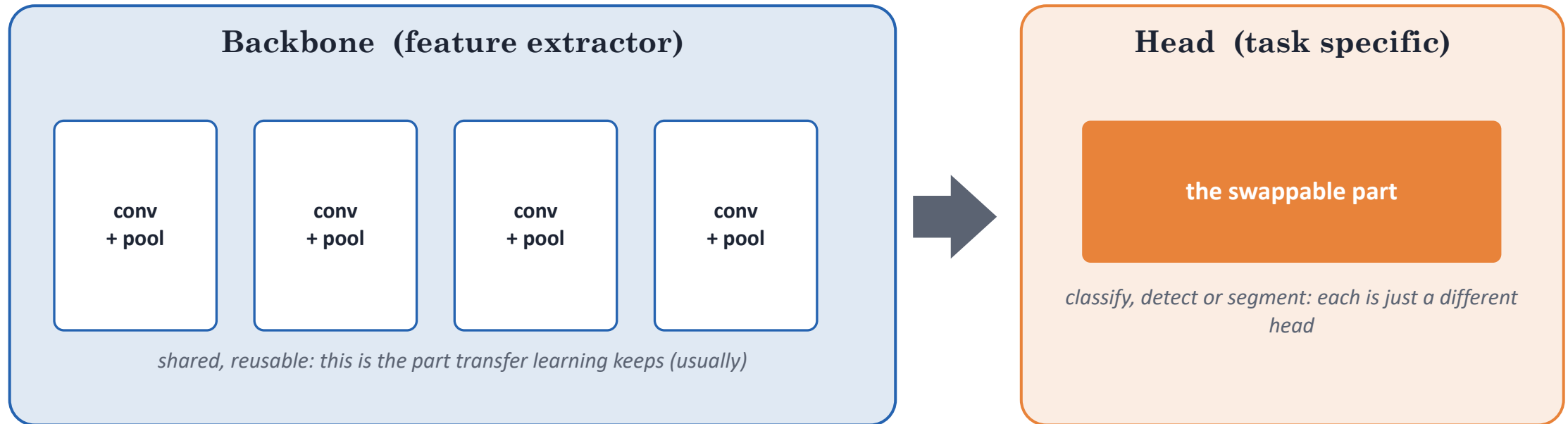
Inside a Vision CNN: How It Is Built

Recap: the building blocks of a CNN



Stack these blocks and a CNN turns raw pixels into useful features. Day 3 builds on top of them.

Every vision CNN has two parts



Train the backbone once to extract features. Swap only the head to change what the network does.

Backbones you will hear about

LeNet 1998

5 layers, read handwritten digits.

AlexNet 2012

Deeper, won ImageNet, started the boom.

VGG 2014

Very deep, all tidy 3×3 filters.

ResNet 2015

Skip connections allow 100+ layers.

Each new backbone learned richer features. Today we usually start from one of these, already trained.

Swap the head, change the question

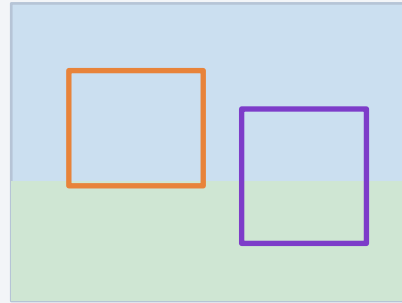
Classification



What is in this picture?

One label for the whole image.

Object detection



What is here, and where?

Boxes around each thing it finds.

Segmentation



Which pixels are what?

A class label for every single pixel.



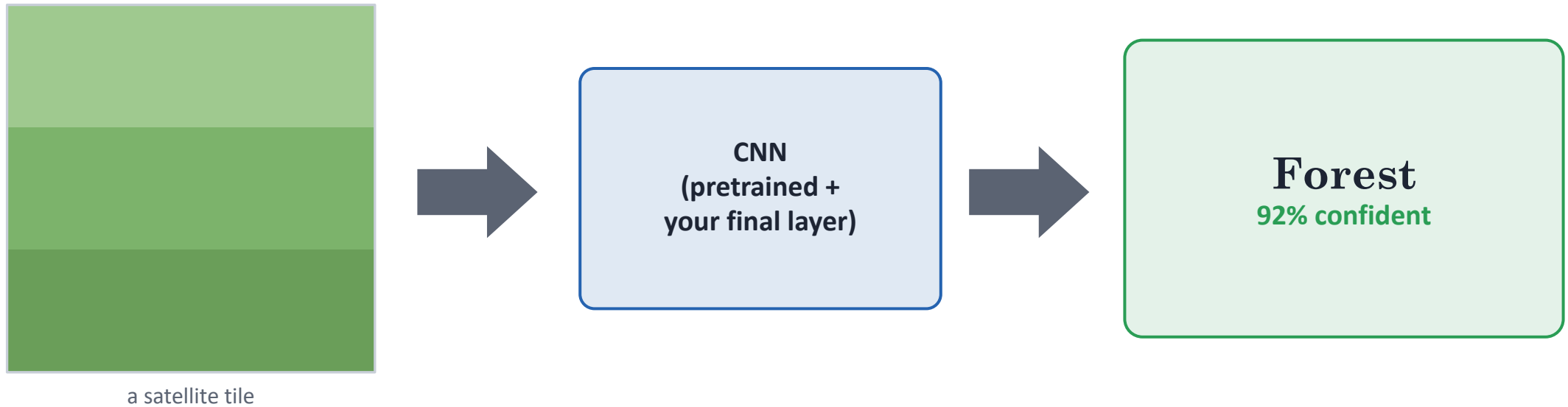
Think for a problem in your field, do you need just a label, a box, or every pixel coloured in?



Part 3

Image Classification: What Is In This Picture?

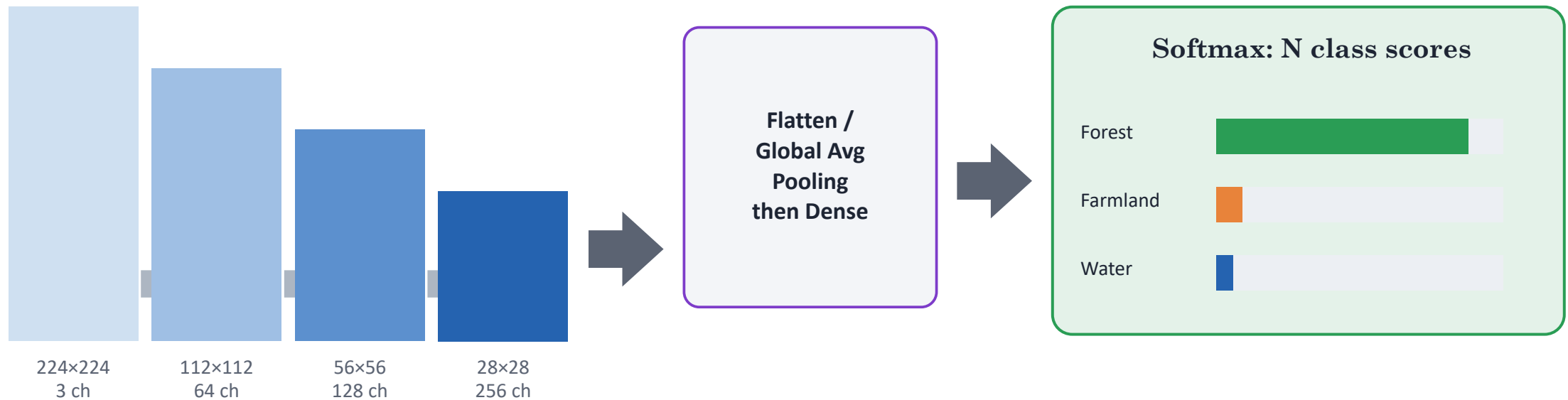
One picture in, one label out



Classification gives the whole image a single name, plus how sure the model is about it.

Designing a classifier: the convolution stack

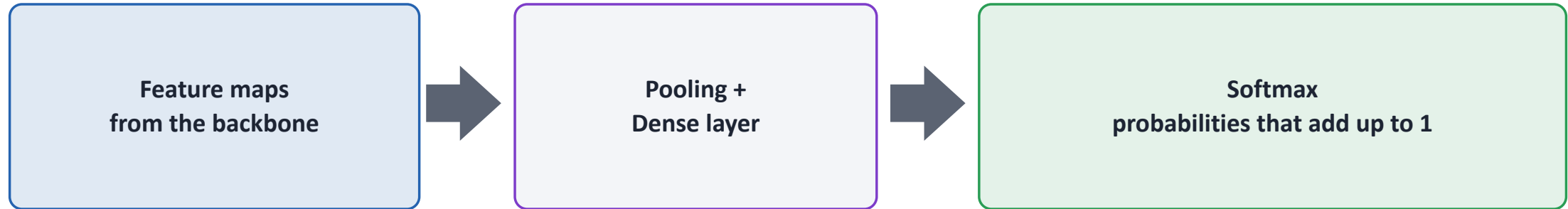
spatial size shrinks → channel depth grows



The convolution stack shrinks the image but grows the channels. A dense layer then turns those features into class scores.



The classifier head and how it learns



Training signal: cross entropy loss. It compares the predicted probabilities with the true label and pushes the correct class towards 1. Backpropagation then nudges every weight to lower that loss.

Design choices for the head: how many dense units, how many output classes, and which loss to minimise.



A good model knows when it is unsure

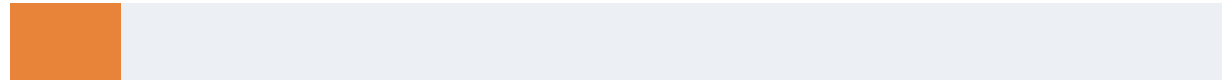
Classification returns a confidence score for every class, not just one hard answer.

Forest



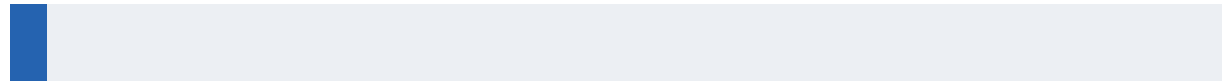
88%

Farmland



9%

Water



3%



Think when would you rather a model say it is only 60% sure than give a confident but wrong answer?

Image classification in your field?



Image classification in your field

Environmental Science

Is this satellite tile forest, water, city or farmland?

Materials Science

Which crystal phase or microstructure is this sample?

Architecture

What architectural style is this building?

Computer Engineering

Does this chip image pass or fail inspection?

Chemical Engineering

Is the reaction clear, cloudy or crystallised?

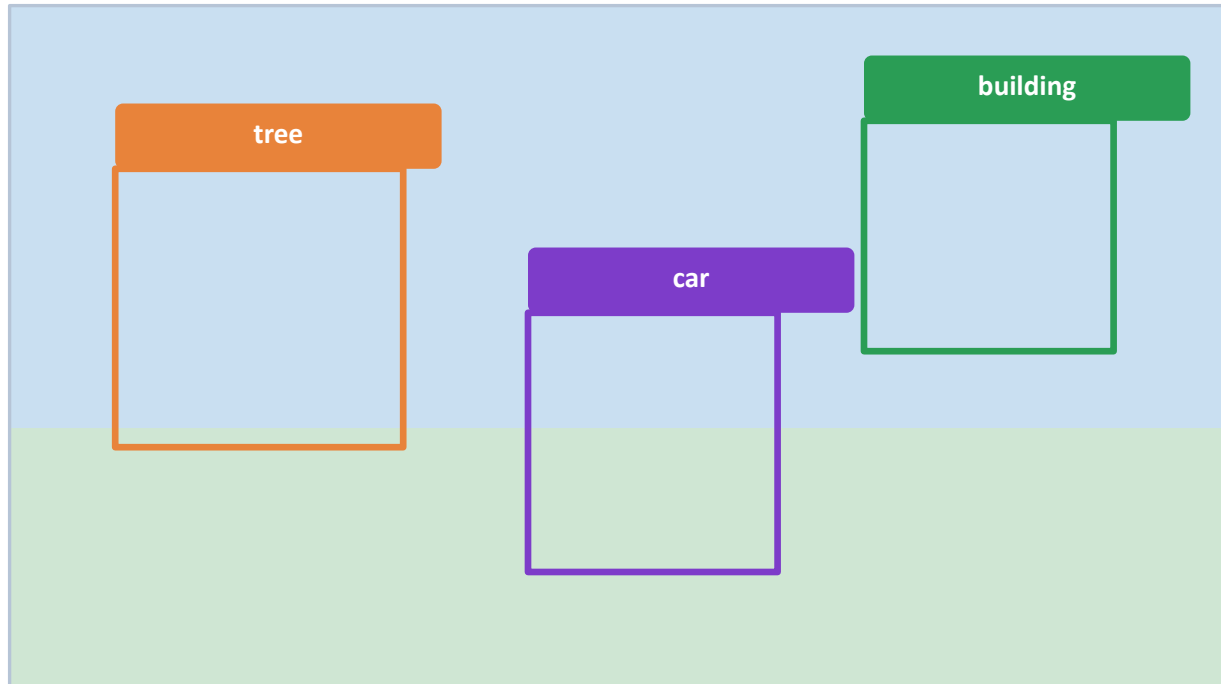
Same idea everywhere: show the model an image, it returns one label from your list of classes.



Part 4

Object Detection: What, and Where?

Detection draws a box around each thing



For each object it returns:

- a class label
- a box: x, y, width, height
- a confidence score
- many objects at once

Detection answers two things at once: what each object is, and where it sits in the image.

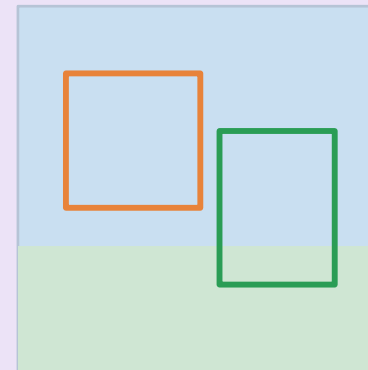
Classification vs detection

Classification



One label for the whole picture. Tells you WHAT, not where.

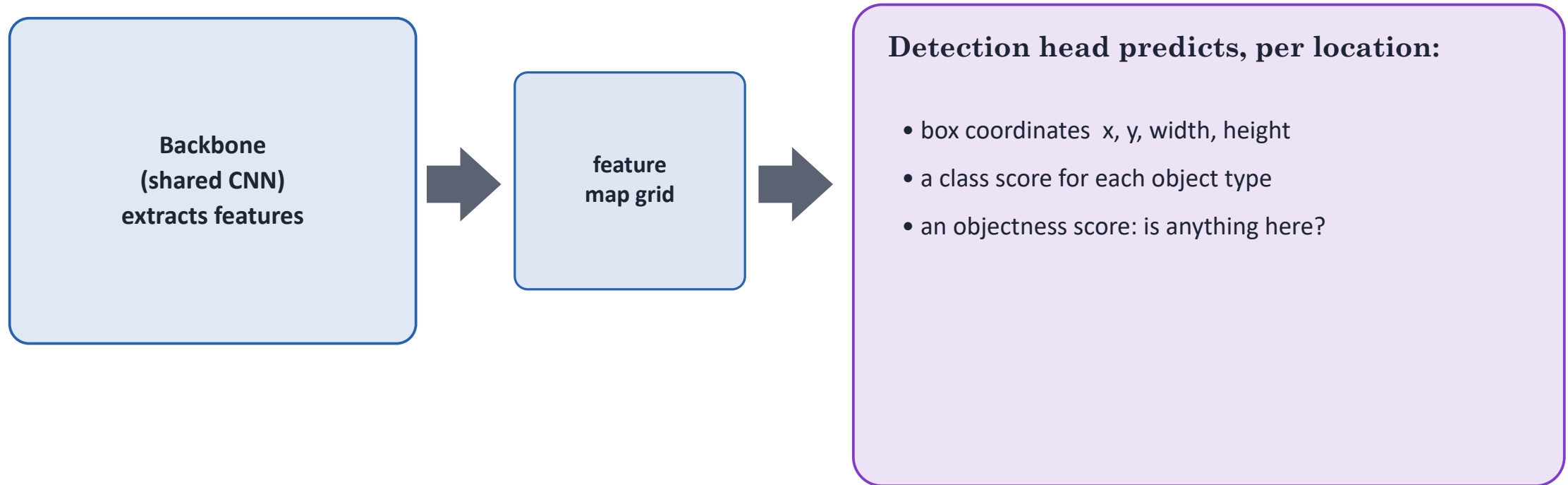
Object detection



Many labelled boxes. Tells you WHAT and WHERE, for every object.

Detection is classification done many times over, each time with a location attached.

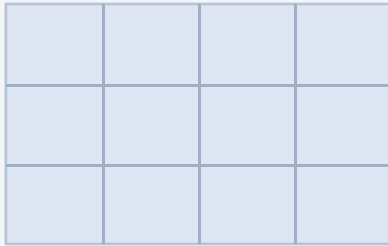
Designing a detector: backbone plus a box head



Same backbone as a classifier, but the head now regresses box positions as well as predicting classes.

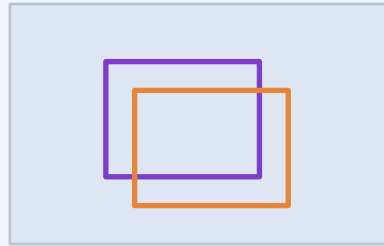
How the boxes are predicted

1 Grid



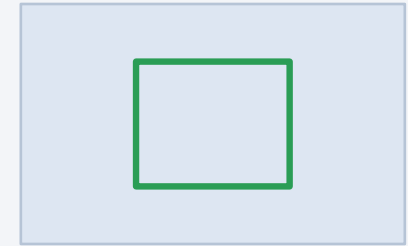
Split the image into cells. Each cell proposes boxes.

2 Anchors



Start from preset box shapes, then adjust to fit.

3 Clean up



Non maximum suppression keeps the best, drops overlaps.

Detectors guess many boxes, score them, then suppress the duplicates so each object keeps one clean box.



Two styles of detector

Two stage (R-CNN family)

First propose regions, then classify each one.
Slower, but very accurate.

One stage (YOLO, SSD)

Predict boxes and classes in a single pass.
Fast enough for live video.



Think *does your application need real time speed, or the highest possible accuracy?*

Object detection in your field?

Object detection in your field

Environmental Science

Count and locate every animal or tree in an aerial photo.

Materials Science

Find and box each crack, pore or inclusion in a micrograph.

Architecture

Locate every window, door and visible crack on a facade.

Computer Engineering

Spot and place each component or solder fault on a board.

Chemical Engineering

Detect and locate bubbles or droplets in a flow image.

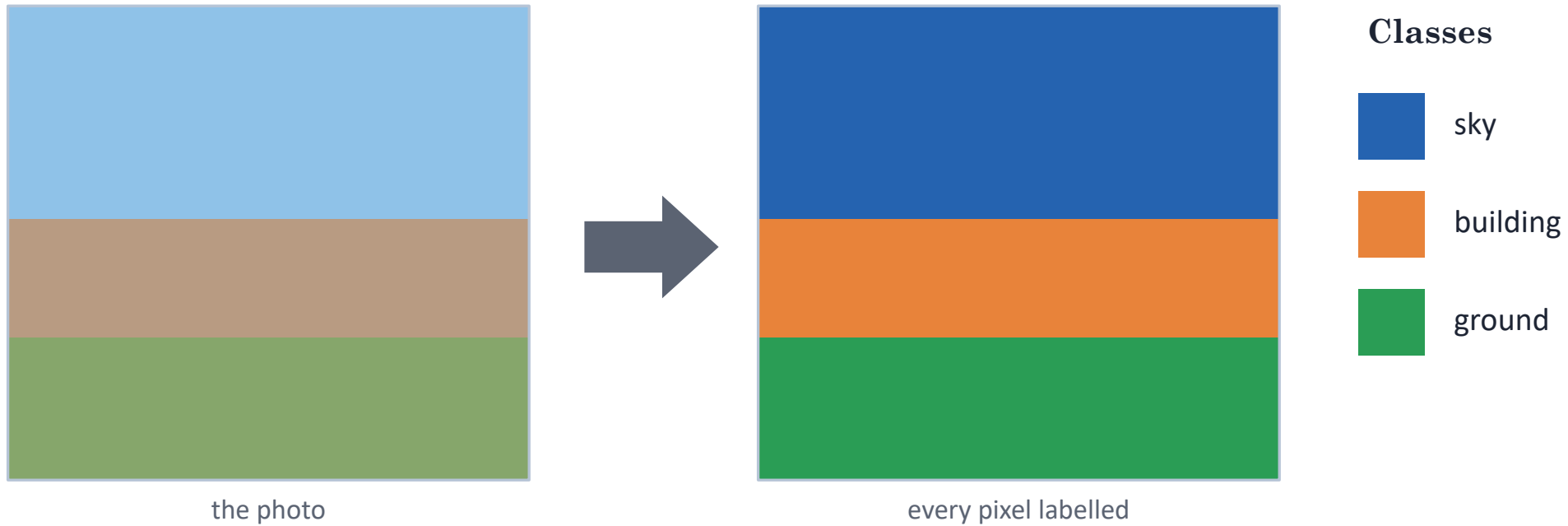
Wherever counting or locating matters, detection gives both the label and the position of each object.



Part 5

Semantic Segmentation: Labelling Every Pixel

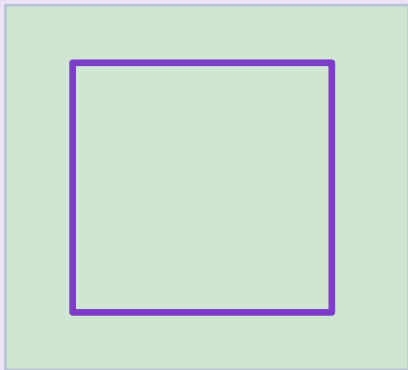
Segmentation colours in every pixel



No boxes. The model returns the exact shape of every region, pixel by pixel, the richest answer of the three.

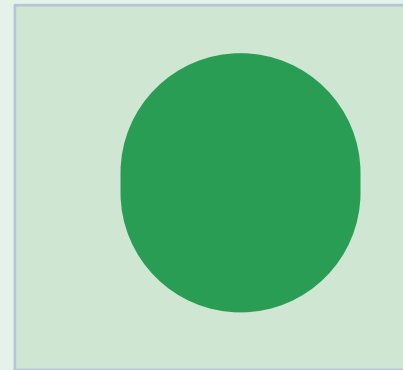
Detection vs segmentation

Detection: a box



A rectangle, roughly where the object is. Fast and simple.

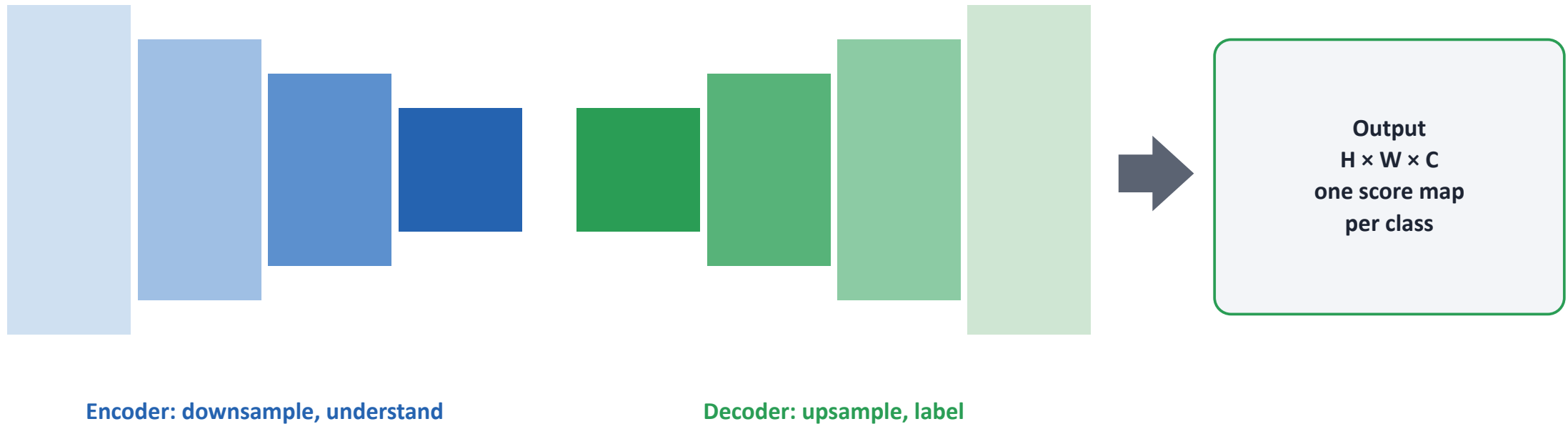
Segmentation: exact shape



The precise outline, useful when area or shape matters.

A box tells you roughly where; a mask tells you the exact pixels. Choose the detail your task needs.

Designing a segmentation network: encoder and decoder



Squeeze the image down to understand it, then expand it back up so the output is the same size as the input.



Getting back to full resolution

Upsampling

Transposed convolutions grow the small feature map back to the full image size, one step at a time.

Skip connections (U-Net)

Copy fine detail from the encoder straight across to the decoder, so edges and small parts stay sharp.



skip detail across →



Skip connections bring back the detail lost during downsampling, which is why U-Net masks look so crisp.

Semantic segmentation in your field?





Semantic segmentation in your field

Environmental Science

Colour each pixel:
flooded, dry land, water
or forest.

Materials Science

Outline every grain and
phase boundary in a
micrograph.

Architecture

Separate walls, windows,
roof and sky in a site
photo.

Computer Engineering

Mark the exact regions of
each trace and
component.

Chemical Engineering

Map the exact area of
each phase in a
multiphase flow.

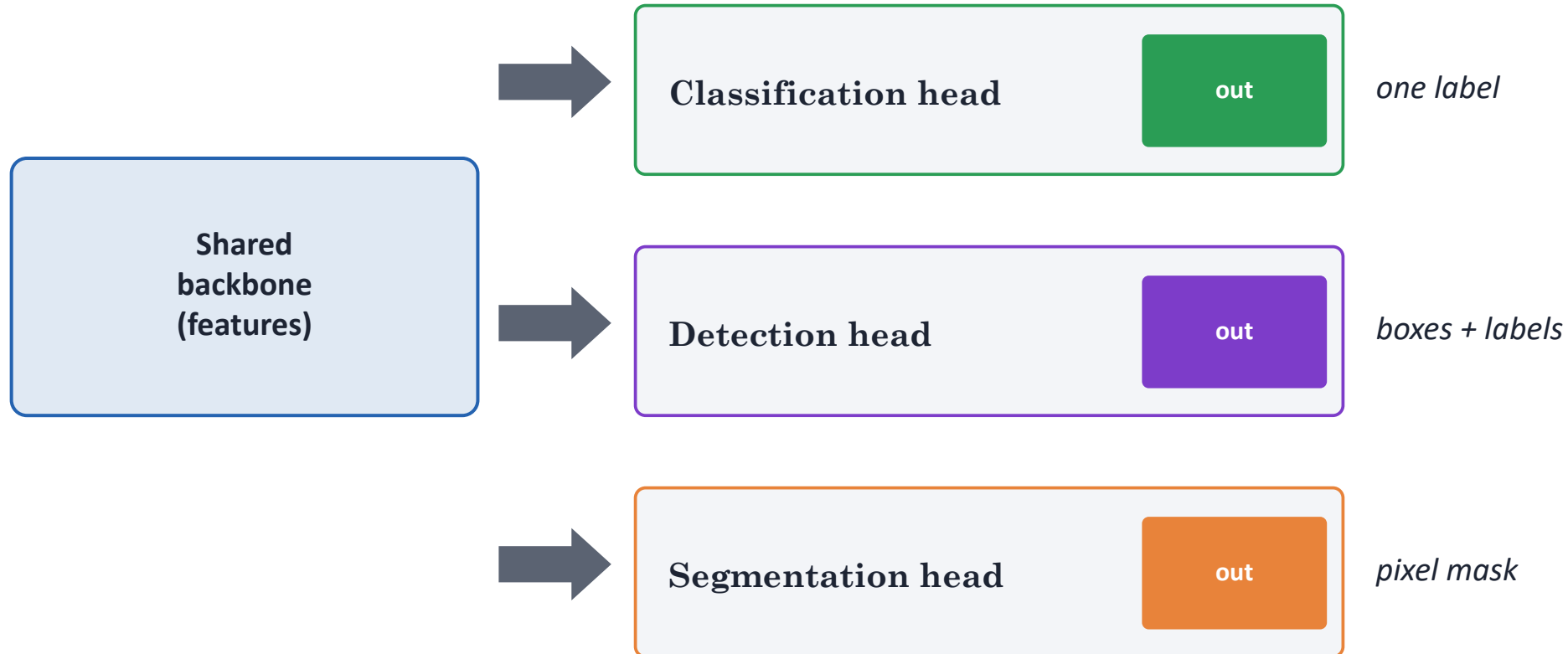


Think *what could you measure if you knew the exact shape and area of every object in an image?*

Part 6

Putting It Together

Same backbone, three different heads



Learn to see once, then attach the head your task needs. That is why transfer learning helps all three.



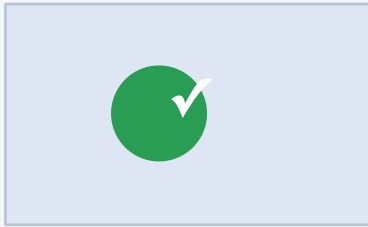
What changes from task to task

	Classification	Object detection	Segmentation
Output	one class label	boxes + labels	a label per pixel
Final layer	dense + softmax	box + class head	1×1 conv per pixel
Loss	cross entropy	box loss + class loss	per pixel cross entropy

The backbone barely changes. What changes is the output shape and the loss that trains the head.

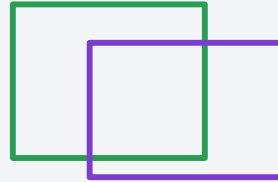
How do we know it works?

Classification



Accuracy: the share of images given the right label.

Object detection



mAP and IoU: how well predicted boxes overlap the true ones.

Segmentation

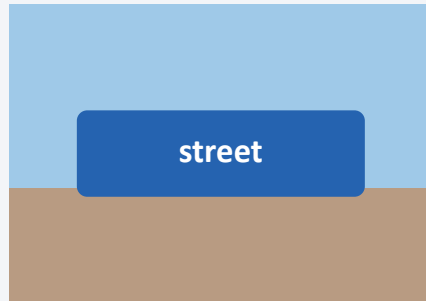


Mean IoU and pixel accuracy: overlap of the predicted mask.

IoU means intersection over union: divide the overlap of two regions by their combined area.

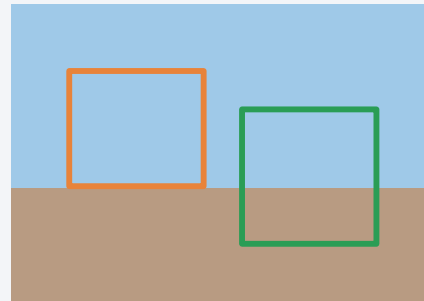
One image, three different answers

Classification



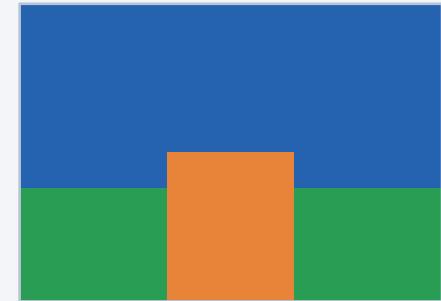
"this is a street"

Detection



"car here, tree there"

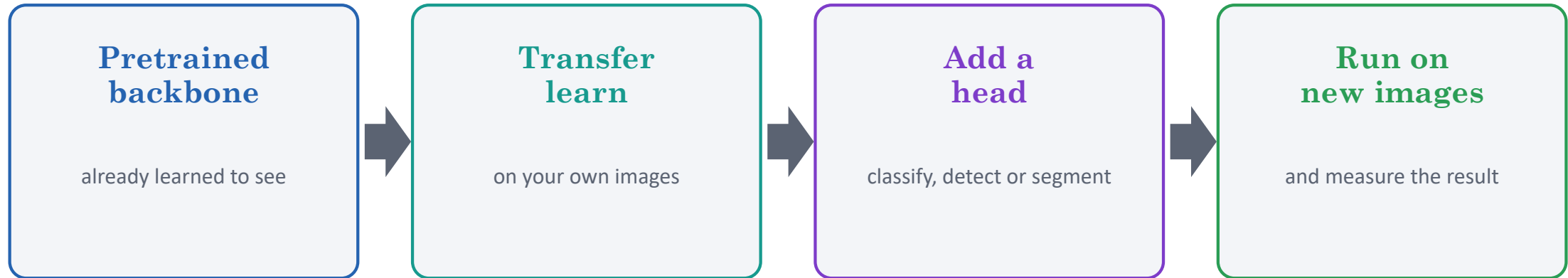
Segmentation



"these pixels are road"

Same picture, same CNN backbone. The difference is the question you ask and the answer shape you want.

How it all fits together



Borrow the seeing, add your labels, pick the task, then check how well it did. That is computer vision in practice.



Key takeaways

1

Transfer learning

Borrow a pretrained backbone and retrain only the head: small data, fast, often free.

2

Backbone + head

Every vision CNN shares a feature extractor; the head decides the task.

3

Classification

Conv stack, then dense and softmax: one label for the whole image.

4

Detection

Backbone plus a box head: what each object is and where it sits.

5

Segmentation

Encoder and decoder: a class for every pixel, the exact shape.

Questions?

Next time: teaching machines to understand language, a first look at Natural Language Processing.